
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON LUNG - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON LUNG

REQUIREMENTS & GRADING CRITERIA

TARGET: NorthPharma HVAC automation node (clean-room air)
ARTIFACT: ACT-IV.bin / ACT-IV.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma moves the air that keeps the medicine cold, and its HVAC automation node built on a Pico 2 is the hand on that air. A contractor called **FROSTLINE** planted an implant in the node image: a reserved-sector persistence marker that re-installs on every boot, a rootkit that masks the beacon from the LCD and the log, a reserved-sector write, and an inverted SETPOINT authorization verdict. Operative **NIGHTINGALE** recovered the compromised image as `ACT-IV.bin`.

Students are the reverse-engineering reserve. They reverse engineer `ACT-IV.bin` with Ghidra, find and patch all four defects, defeat the CoreDebug `DHCSR` anti-debug under GDB to observe the persistence write, export a corrected image, flash it to a real Pico 2, and prove the corrected behavior on the breadboard. The machine check is `scripts/verify_ctf.py`.

The challenge is a standalone capstone exercise and contains no answer, constant, address, bug, or patch belonging to any other course assignment.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the monitor loop.
- Locate four corrupted bytes: a boot re-install gate, a rootkit masking gate, a reserved-sector persist gate, and an authorization verdict branch.
- Analyze `cbz`, `cbnz`, `beq`, and `bne` condition semantics and branch inversion.
- Explain why reserved-flash state survives a firmware reflash and why a rootkit controls visibility rather than hiding code.
- Read CoreDebug `DHCSR`, explain the anti-debug trap, and defeat it under GDB.
- Explain why authentication is not authorization and why a verdict must be verified before the setpoint is applied.

Students must use only the course concepts: ARM registers, stack behavior, USB-CDC and UART consoles, GDB, Ghidra static analysis and binary patching, vector tables, reset startup, XIP, Thumb addressing, condition-code analysis, stateful security, and the Argon2id plus XChaCha20-Poly1305 authenticated envelope.

Deliverables Checklist

#	Deliverable	Format	Criterion
1	Ghidra project screenshot	PNG/JPG	Task 1
2	Vector table and boot table	Inside <code>ACT-IV-Answers.md</code>	Task 1
3	<code>main</code> and monitor-loop table	Inside <code>ACT-IV-Answers.md</code>	Task 1
4	Module map	Inside <code>ACT-IV-Answers.md</code>	Task 1
5	Persistence re-install evidence and patch	Inside <code>ACT-IV-Answers.md</code>	Task 2
6	Rootkit hiding evidence and patch	Inside <code>ACT-IV-Answers.md</code>	Task 3
7	Anti-debug GDB proof, reserved-sector evidence, and patch	Inside <code>ACT-IV-Answers.md</code>	Task 4
8	SETPOINT authorization evidence and patch	Inside <code>ACT-IV-Answers.md</code>	Task 5
9	<code>ACT-IV_fixed.bin</code>	BIN file	Task 6
10	<code>ACT-IV_fixed.uf2</code>	UF2 file	Task 6
11	Hardware proof and reflection	Inside <code>ACT-IV-Answers.md</code>	Task 6

Required Tools and Equipment

Tool	Purpose
Raspberry Pi Pico 2	Isolated target node
Debug Probe (OpenOCD)	SWD connection for GDB inspection and the anti-debug work
arm-none-eabi-gdb	Runtime breakpoints, <code>DHCSR</code> clearing, and reserved-sector observation
Ghidra	Static analysis and binary patching
Python 3 with <code>uf2conv.py</code>	UF2 conversion and artifact checks
DHT11, 1602 I2C LCD, RYL998, IR receiver, SG90 servo, 3 LEDs, override button	Breadboard hardware proof
<code>ACT-IV.bin</code> and <code>ACT-IV.uf2</code>	Supplied compromised artifacts

Console settings: **USB-CDC virtual COM port, 115200 baud, 8 data bits, no parity, 1 stop bit.** Radio UART settings: **UART1, 115200, network ID 18.**

Artifact Identity

The instructor-issued artifact hashes are:

```

ACT-IV.bin          efc9991c3114d51994aa0dc080bbcb592bc9432c7739b850eeb63e73e7e62316
ACT-IV.uf2          f7e79d0b0e3104e3ee0f3a2c032d9ba95da2d1128235ffa8b822d74c6b8542be
ACT-IV_fixed.bin    7d59dc65e512c30630f51b20baf1c6e358ae54a024e88cfd8373d847ea170f7c
ACT-IV_fixed.uf2    977c4cad1f60e05060fd933050ecc7c0f04e319cbee9c3c441c41c9e37137775

```

The verifier checks the `ACT-IV.bin` and `ACT-IV_fixed.bin` hashes specifically, asserts the four fixed bytes, and requires that only those four offsets differ between the two `.bin` images. Both `.bin` images are 51,012 bytes and both `.uf2` images are 102,912 bytes.

Grading Rubric - Detailed Breakdown

Task 1: Setup and Initial Analysis (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronLung_Investigation</code> , raw binary import	One item off	Not set up
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor	Wrong language	Missing
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>	Wrong base	Missing
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>	One missing	Not found
[DOCUMENT] main and the HVAC controller state machine (<code>monitor_step</code>) addresses identified	1	<code>main</code> <code>0x10000234</code> , <code>monitor_step</code> <code>0x100065E0</code>	One correct	Neither
[DOCUMENT] Module map identifies the baffle, control, hvac_auth, implant, and monitor anchors	1	At least one correct anchor per module	Partial	Missing

Task 2: Bug #1 The Persistence Re-Install (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the persistence re-install branch at 0x1000A46F	5	Address and function (<code>implant_init</code>) identified	Approximate	Not found
[DOCUMENT] Documented the reserved-sector marker and the re-install branch	5	<code>0x103FF000</code> , marker <code>0xC7</code> , a present marker re-arms on boot	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so a present marker no longer re-arms the implant	7	Byte <code>0xD1</code> changed to <code>0xD0</code>	Wrong byte	Not patched
[DOCUMENT] Explained that a firmware reflash alone does not remove the implant	3	The reserved sector is outside the program region	Vague	Missing

Task 3: Bug #2 The Rootkit Hiding (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the rootkit hiding branch at 0x1000A2D1	5	Address and function (<code>implant_rootkit_active</code>) identified	Approximate	Not found
[DOCUMENT] Documented the LCD and log masking (<code>B:UP</code> to <code>B:--</code> , <code>BCN</code> suppressed) while the beacon transmits	5	Correct masking detail	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the rootkit gate is closed	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained how to prove the beacon status is visible again	3	<code>B:UP</code> and the <code>BCN</code> line return while the rootkit gate is clear	Vague	Missing

Task 4: Bug #3 The Reserved-Sector Write (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the reserved-sector persist branch at 0x1000A42D	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so no marker is written to 0x103FF000	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained the reserved sector 0x103FF000 and the write-once marker byte 0xC7	3	Marker, reserved sector, write-once first run	Vague	Missing

Task 5: Bug #4 The SETPOINT Authorization (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the SETPOINT authorization branch at 0x10007581	5	Address and function (<code>control_handle_frame</code>) identified	Approximate	Not found
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so unauthenticated and replayed commands are rejected	7	Byte <code>0xD1</code> changed to <code>0xD0</code>	Wrong byte	Not patched

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Explained that unauthenticated and replayed SETPOINT commands must be rejected	3	The actuator decision must see only an authorized verdict	Vague	Missing

Task 6: Export and Verify (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[PATCH] Exported ACT-IV_fixed.bin from Ghidra	2	Valid patched binary	Corrupt	Not submitted
[PATCH] Converted to ACT-IV_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59	Wrong flags	Not submitted
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown	Partial proof	No proof
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four	Partial	Missing

Common Pitfalls

Pitfall	Consequence	Avoidance
Reading the re-install gate backwards	A present marker still re-arms the bomb on boot	Accept only on the clear-gate branch (beq , 0xD0)
Confusing cbz and cbnz at 0xA2D1 or 0xA42D	The rootkit still hides the beacon, or the marker is still written	Neutralize only when the gate is clear (cbz , 0xB1)
Searching for a standalone implant_persist symbol	Cannot find the marker gate	Look inside implant_init at 0x1000A42D
Patching the shipped image before observing the write	You never prove the marker write	Defeat DHCSR under GDB first, then patch the artifact
Fabricating the GDB session	Verification fails	Show the command sequence and the real observed code path
Treating the anti-debug as a defect to patch	Wasted effort; it is identical in both images	Defeat it in a scratch copy or with GDB, then patch the real defect
Missing that the authorization branch is a verdict	Unauthenticated commands still reach the setpoint	Accept only when the verdict is true (beq to reject, 0xD0)
Forgetting UF2 conversion	Raw binary will not flash	Use uf2conv.py with family 0xe48bff59

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 room climate sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HVAC ALARM, 220 to 330 ohm to GND
Yellow LED	anode	GP17	OVERRIDE PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	HVAC NOMINAL, 220 to 330 ohm to GND
Manual override button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE00EDF0</code>	Anti-debug register read by the implant
Implant reserved sector	<code>0x103FF000</code>	Persistence marker target (sector)
Implant tick counter	<code>0x20013710</code>	Incremented once per <code>implant_tick</code>
Implant trigger tick	<code>0x20013714</code>	Tick at which an armed bomb detonates

Region	Address	Purpose
Implant arming flag	0x20013CF7	Set when FR0STLNE arms the bomb
Implant persist gate	0x20013CF8	Gates the reserved-sector marker write
Implant re-install gate	0x20013CF9	Gates the boot re-install
Implant rootkit gate	0x20013CFA	Gates the LCD and log masking
SETPOINT command gate	0x20013CF3	Gates the sealed SETPOINT path
Applied setpoint	0x20013CE6	Setpoint written after a true verdict
Auth state record	0x200136AC	Anti-replay and state-tag record
Auth field key	0x200136C8	Derived field key for the tag

The VA of any file offset is the file offset plus 0x10000000.

Deadline & Submission

- Create a folder containing the Ghidra screenshot, ACT-IV_fixed.bin, and ACT-IV_fixed.uf2.
- Write all written answers in ACT-IV-Answers.md inside that folder.
- Include the output of python scripts/verify_ctf.py.
- ZIP the folder as lastname-firstname-ACT-IV.zip.
- Submit the ZIP before the posted deadline; late submissions lose 10 percent per day.

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-69%	0-69

Academic Integrity

Use only the supplied Pico 2 and firmware. Do not connect the exercise to an operational building-management system, a clean-room control system, a pharmaceutical network, a public network, a military system, or a third-party device. This is a controlled, isolated educational exercise. All analysis and patches must be your own work; sharing binaries, addresses, keys, passphrases, or answers is a violation of the academic integrity policy.

Reference Material

Topic	Reference
ARM Cortex-M33 registers and stack	Course block 1
USB-CDC and UART console capture	Course block 2
Vector tables, reset startup, and XIP	Course block 3
Ghidra static analysis and binary patching	Course block 4
Reserved-flash persistence and boot re-install	Course block 5
Rootkits, visibility control, and covert beacons	Course block 6
CoreDebug DHCSR and anti-debug	Course block 7
Argon2id and XChaCha20-Poly1305 authenticated envelope	Course block 8
Authentication versus authorization and fail-safe policy	Course block 9